

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Constraint Based Problem Solving

Code of Conduct:

*I Moshitha kolanjiyappan (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Moshitha k

Annexure A

Short Answer Test

1, You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

A large patient health dataset may contain missing values, inconsistent formats, duplicate records, and medically invalid values. A constraint-based cleaning pipeline can be designed using **Pandas and NumPy** to improve data quality while preserving as much valid information as possible.

First, the CSV file is loaded using `pd.read_csv()`. The structure of the dataset is inspected using `df.info()`, `df.shape`, `df.head()`, and `df.isnull().sum()`. Numerical columns such as age, weight, height, blood pressure, and glucose level are converted to appropriate numeric types using `pd.to_numeric()` with invalid values converted to NaN.

Missing values are handled according to the meaning of each field. Numerical missing values can be replaced using median values calculated from suitable groups, while categorical missing values can be replaced with the most frequent valid category or an explicit "Unknown" category. The objective is to avoid blindly deleting records because this could result in significant data loss.

Medical constraints are enforced using filtering. For example, age may be restricted to a reasonable range such as 0–120 years, while systolic blood pressure, BMI, glucose, and other measurements are checked against predefined valid ranges. Invalid values can either be corrected when the original value is clearly recoverable or marked as missing and subsequently imputed.

Categorical fields such as gender, smoking status, and activity level should be standardized using `.str.strip()`, `.str.lower()`, and mapping dictionaries. For example, "Male", "male", and "M" can be mapped to "Male".

Indexing is used to select relevant rows and columns, while aggregation can calculate medians, minimums, maximums, and counts for validation. After cleaning, assertions and summary statistics should verify that no missing values remain and all constraints are satisfied.

2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to

minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

When processing a massive numerical dataset with limited memory, the primary objective is to minimize unnecessary copies of arrays. NumPy arrays should be created with appropriate data types using dtype, such as float32 instead of float64 when the required precision allows it.

Instead of loading an entire dataset into memory, data can be processed in chunks. If the complete dataset is already available as an array, **slicing** should be preferred because basic slicing generally creates a view rather than a complete copy.

Vectorized NumPy operations should be used instead of Python loops because they are faster and generally more efficient. In-place operations such as `arr *= 2`, `arr += value`, and `arr /= scale` can reduce additional memory allocation.

Random sampling can be performed using NumPy's random-generation functions, selecting only the required sample size instead of copying the complete dataset. Aggregation functions such as `np.mean()`, `np.sum()`, `np.min()`, and `np.max()` can operate directly on arrays.

Indexing should be carefully designed to avoid unnecessary advanced-indexing copies. If memory overflow occurs, debugging should involve checking `array.nbytes`, data types, array dimensions, and the number of temporary arrays created. Performance bottlenecks can be investigated using timing tools and by comparing vectorized operations with loops.

3. Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

Two large Pandas DataFrames containing customer transaction and demographic information can be combined using a common **Customer ID**. Before merging, both datasets should be inspected to determine whether Customer IDs are unique and whether their data types are consistent.

The ID columns should be converted to a common format, for example using `astype("string")`. Duplicate customer IDs should be identified using `duplicated()` and investigated. If the demographic dataset is expected to contain one record per customer, uniqueness can be enforced using:

```
df["Customer_ID"].is_unique
```

The appropriate merge operation depends on the requirement. An inner join retains only customers appearing in both datasets, while a left join preserves every customer in the primary DataFrame. If preservation of all valid records from both datasets is required, an outer join can be used.

The `indicator=True` option in `merge()` is useful for debugging because it identifies records appearing only in the left DataFrame, only in the right DataFrame, or in both.

After merging, grouping by Customer ID can detect unexpected duplicates:

```
merged.groupby("Customer_ID").size()
```

Sorting by Customer ID and resetting the index using `reset_index(drop=True)` ensures a clean and consistent final index.

Missing IDs and unmatched records should be examined rather than automatically deleted. This approach helps preserve valid information and improves data integrity.

4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

Statistical measures such as mean, median, and variance can be calculated using both NumPy and Pandas. The first step is to identify relevant numerical columns and remove or handle invalid values.

For example, NumPy provides `np.mean()`, `np.median()`, and `np.var()`, while Pandas provides `df.mean()`, `df.median()`, and `df.var()`. Pandas functions generally handle missing values using NaN-aware behavior by default.

Filtering should be performed before statistical calculations to remove irrelevant or invalid observations. For example, records outside a valid measurement range can be excluded using Boolean indexing.

To improve computational efficiency, unnecessary conversions between Pandas and NumPy should be avoided. If multiple statistics are required, aggregation can calculate them together instead of repeatedly scanning the same dataset.

Numerical accuracy is also important. NumPy's appropriate data types should be selected based on the required precision. For highly sensitive calculations, `float64` can be retained instead of using lower-precision types.

Incorrect outputs can be debugged by checking the number of valid observations, missing values, minimum and maximum values, and the selected filtering conditions. Comparing Pandas and NumPy results on a small sample can also help identify implementation errors.

5. Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing,

aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

A complete data analysis pipeline consists of **loading, cleaning, transforming, analyzing, and visualizing data**.

First, multiple files can be discovered and loaded using Pandas. CSV files can be read using `pd.read_csv()`, while other formats can use appropriate Pandas functions. When files are very large, only required columns can be loaded using `usecols`, and data can be processed in chunks using `chunksize`.

After loading, each DataFrame should be checked for column names, data types, missing values, and duplicate records. Inconsistent column names should be standardized. Numerical columns should be converted to numeric formats, and categorical values should be normalized.

Filtering can be performed according to user-defined conditions. Grouping and aggregation can then calculate statistics such as totals, averages, counts, and percentages. Sorting makes the results easier to interpret.

The processed data can be visualized using Matplotlib. Bar charts are suitable for category comparisons, line charts for time-series trends, histograms for distributions, and scatter plots for relationships between numerical variables.

Corrupted files should be detected using exception handling. Schema validation should ensure that required columns exist before processing. Debugging should include checking row counts before and after every major transformation so that unexpected data loss can be detected.

6. You are given a large dataset of financial transactions containing duplicate records, inconsistent timestamps, and invalid entries. Design a data cleaning pipeline using NumPy and Pandas with constraints: (i) remove duplicates without losing valid data, (ii) standardize time formats, and (iii) filter out invalid transactions. Explain how indexing, grouping, and sorting ensure data integrity.

Financial transaction data can contain duplicates, inconsistent timestamps, missing values, and invalid transactions. A cleaning pipeline should first load the data using Pandas and inspect its structure.

Duplicate transactions can be identified using `duplicated()`. However, duplicates should not be removed blindly. A transaction identifier or combination of fields such as account ID, timestamp, amount, and transaction type should be used to determine whether records are genuinely duplicated.

Timestamps should be converted using `pd.to_datetime()`. Different formats are thereby converted into a consistent datetime representation. Invalid timestamps can be identified using `errors="coerce"`.

Invalid financial transactions can be filtered using logical conditions. For example, negative transaction amounts can be rejected if the business rules do not permit them. Required account IDs and transaction IDs should not be missing.

Grouping can be used to detect suspicious duplicate patterns, such as repeated transactions with the same identifier. Sorting by timestamp and transaction ID helps establish a consistent order.

Finally, the index should be reset using `reset_index(drop=True)`. Summary statistics before and after cleaning should be compared to ensure that valid data has not been accidentally removed.

7. A real-time sensor system generates streaming data that must be processed under strict latency constraints. Using NumPy arrays, design a solution that performs filtering, normalization, and anomaly detection efficiently. Ensure minimal computation delay and memory usage. Discuss how slicing, vectorization, and aggregation functions can be optimized, and explain debugging techniques for delayed processing or incorrect outputs.

A real-time sensor system requires low latency and efficient memory usage. NumPy is suitable because it provides fast vectorized numerical operations.

Incoming sensor measurements can be stored in a fixed-size NumPy buffer instead of continuously creating new arrays. Slicing can extract the latest observations without processing the entire history.

Filtering can remove measurements outside valid sensor ranges. Normalization can be performed using vectorized expressions such as subtracting the mean and dividing by the standard deviation.

Anomaly detection can use statistical thresholds. For example, values more than a specified number of standard deviations from the mean can be flagged as anomalies. Aggregation functions such as mean, std, min, and max can summarize recent measurements.

To reduce latency, Python loops should be minimized. Vectorized NumPy operations and fixed-size buffers are preferred. Processing only the newest time window also reduces computation.

Debugging should monitor processing time, buffer size, input frequency, and output frequency. Incorrect results can be investigated by testing the algorithm with known synthetic sensor values. Delayed processing may indicate unnecessary copies, large windows, inefficient loops, or excessive logging.

8. You are required to merge multiple large datasets containing product information from different sources. Constraints include: (i) resolving conflicting attribute values, (ii) maintaining consistent schema, and (iii) avoiding data loss. Describe how you will use Pandas merging, grouping, and filtering techniques to enforce constraints and validate the final dataset.

When product information comes from different sources, the first step is to standardize the schema. Column names, data types, units, and categorical values should be made consistent.

Product IDs should be converted to the same data type and checked for duplicates. DataFrames can then be merged using Pandas `merge()` based on a common Product ID.

Conflicting attributes such as price, product description, or category may appear in multiple sources. A predefined priority rule should determine which source is trusted. For example, the most recent verified record may be selected.

Grouping by Product ID helps detect multiple records for the same product. Filtering can identify conflicting values before they are resolved.

An outer merge can initially be used when avoiding data loss is important. Records appearing only in one source should be reviewed instead of immediately discarded.

After conflict resolution, the final DataFrame should be sorted by Product ID and given a clean index. Row counts and unmatched records should be compared with the original datasets to validate that no unexpected data has been lost.

9. A dataset contains millions of records with mixed data types and must be transformed into a structured format suitable for machine learning. Constraints include: (i) efficient type conversion, (ii) handling missing and categorical data, and (iii) ensuring consistent indexing. Explain how you will use Pandas operations and NumPy functions to preprocess data while maintaining computational efficiency.

Large mixed-type datasets require efficient preprocessing before machine learning. Pandas can be used for data organization, while NumPy supports numerical operations.

The first step is to inspect data types using `df.dtypes`. Numerical columns should be converted using `pd.to_numeric()`, while dates should be converted using `pd.to_datetime()`.

Missing numerical values can be replaced using suitable statistics such as the median. Categorical missing values can be represented using a suitable category or imputed according to the application.

Categorical variables must be encoded before machine learning. Depending on the algorithm, techniques such as one-hot encoding or categorical codes can be applied.

Unnecessary columns should be removed early to reduce memory consumption. Appropriate data types such as `int32`, `float32`, and Pandas categorical types can reduce memory usage.

The index should be reset after major filtering operations. Vectorized Pandas and NumPy operations should be used rather than row-by-row loops. Finally, data types, missing values, duplicate rows, and index uniqueness should be checked before passing the dataset to the machine-learning model.

10. You need to compute rolling statistics (moving average, rolling variance) on a time-series dataset under constraints of high performance and accuracy. Describe how to implement rolling window operations using Pandas and NumPy, ensuring efficient indexing and minimal

redundancy. Discuss debugging strategies for incorrect window alignment and performance bottlenecks.

Rolling statistics are useful for analyzing time-series data. Pandas provides efficient rolling-window operations through `.rolling()`.

The data should first be sorted by timestamp. For example, a moving average can be calculated using:

```
df["Moving_Average"] = df["Value"].rolling(window=7).mean()
```

Similarly, rolling variance can be calculated using `.var()`.

Correct indexing is important because the order of observations determines the rolling window. If the data is not sorted chronologically, the results can be incorrect.

For time-based windows, Pandas can use time offsets when the DataFrame has a proper datetime index. This is useful when observations occur at irregular intervals.

Performance can be improved by calculating only the required rolling measures and avoiding repeated transformations. NumPy can be useful for specialized numerical operations, while Pandas provides convenient time-series indexing.

Debugging should involve manually calculating the first few windows and comparing them with the program output. Problems such as incorrect window size, timestamp ordering, and unexpected NaN values should be checked. Performance bottlenecks can be investigated by measuring execution time and reducing unnecessary columns or repeated calculations.

11. You are given a large healthcare dataset with mixed structured and semi-structured data (CSV and JSON formats). Design a preprocessing pipeline using **NumPy and Pandas** with constraints: (i) unify different data formats into a single schema, (ii) handle nested JSON fields efficiently, and (iii) ensure no data inconsistency after transformation. Explain how indexing, normalization, and aggregation techniques are used, and describe debugging strategies for schema mismatches.

Healthcare datasets may contain structured CSV records and semi-structured JSON records. The objective is to transform them into a unified schema without introducing inconsistencies.

CSV data can be loaded using `pd.read_csv()`, while JSON data can be processed using `pd.read_json()` or JSON normalization functions. Nested JSON structures can be flattened using `pd.json_normalize()`.

Column names and data types should be standardized before combining the datasets. For example, patient IDs must have the same data type and format.

Nested fields such as patient address, medical history, or laboratory results may need to be extracted into separate columns. Missing fields should be handled consistently.

Indexing can ensure that patient records are uniquely identified. Grouping by patient ID can detect multiple records and support aggregation of repeated measurements.

Normalization may involve converting units, standardizing categorical values, and transforming dates into a common format.

Schema validation should compare expected and actual columns. Debugging can include checking records before and after JSON flattening, identifying missing columns, and comparing row counts. This helps ensure that no important healthcare information is lost during transformation.

12. A financial analytics system must compute real-time risk metrics from streaming stock market data under strict latency constraints. Using **NumPy**, design a solution that performs vectorized computations for returns, volatility, and correlations. Ensure minimal memory usage and fast execution. Explain how slicing, broadcasting, and aggregation functions are optimized, and discuss debugging techniques for incorrect calculations or delayed outputs.

A financial analytics system can use NumPy to calculate stock returns, volatility, and correlations efficiently.

For a sequence of prices, returns can be calculated using vectorized operations. For example, percentage returns can be computed using:

```
returns = prices[1:] / prices[:-1] - 1
```

This avoids Python loops and significantly improves performance.

Volatility can be calculated from the standard deviation of returns. For example:

```
volatility = np.std(returns)
```

For multiple assets, the return matrix can be stored as a two-dimensional NumPy array, where rows represent time points and columns represent assets. Correlations can then be calculated using NumPy statistical functions.

Slicing allows only the most recent observations to be processed, reducing memory and computation requirements. Broadcasting can apply operations across multiple assets without creating unnecessary repeated arrays.

For real-time systems, calculations should operate on a fixed-size rolling window. Intermediate arrays should be minimized and appropriate numerical precision should be maintained.

Debugging should verify return calculations using manually calculated examples. Unexpected correlations or volatility values should be investigated by checking missing values, price ordering, data frequency, and numerical precision. Latency can be monitored to identify slow operations.

13. You are tasked with cleaning a large dataset of social media posts containing text, timestamps, and user metadata. Constraints include: (i) removal of duplicate and spam entries, (ii) normalization of text data, and (iii) consistent timestamp formatting. Describe how you will use **Pandas operations** such as filtering, grouping, and string processing to enforce these constraints, and explain debugging strategies for handling noisy and inconsistent data.

Social media datasets often contain duplicate posts, spam, inconsistent text, timestamps, and user metadata. Pandas provides suitable tools for cleaning these records.

Duplicate posts can be identified using `duplicated()`, using post IDs or combinations of user ID, timestamp, and content. Spam can be identified using filtering rules, such as repeated posts, excessive links, or known spam indicators.

Text normalization can be performed using Pandas string operations. For example, `.str.lower()` converts text to lowercase, `.str.strip()` removes unnecessary spaces, and `.str.replace()` can remove unwanted patterns.

Timestamps should be converted using `pd.to_datetime()` and standardized into a consistent format. Invalid timestamps can be identified and handled separately.

Grouping by user ID can help identify users generating unusually high numbers of posts. Sorting by timestamp can establish chronological order.

Debugging noisy data requires inspecting representative samples before and after each transformation. Counts of records should be compared after duplicate and spam removal. Regular checks should ensure that legitimate posts are not incorrectly classified as spam.

14. A machine learning pipeline requires feature extraction from a high-dimensional dataset under constraints of computational efficiency and scalability. Using **NumPy and Pandas**, design a solution that performs feature selection, normalization, and dimensionality reduction. Explain how indexing, vectorization, and aggregation are used to optimize performance, and describe debugging steps for detecting redundant or irrelevant features.

A machine-learning dataset with many features may contain irrelevant, redundant, or highly correlated variables. Feature selection reduces computational cost and can improve model performance.

Pandas can be used to inspect columns, calculate missing-value percentages, and identify duplicate features. Numerical features can be analyzed using correlation matrices and descriptive statistics.

Indexing can select only relevant columns, while filtering can remove features with excessive missing values or extremely low variance.

Normalization is important when features have different scales. NumPy can perform vectorized operations for standardization, such as subtracting the mean and dividing by the standard deviation.

Dimensionality reduction methods can be applied after preprocessing. A common approach is Principal Component Analysis (PCA), which transforms correlated features into a smaller number of components.

Vectorization avoids expensive row-by-row processing. Aggregation functions can summarize feature statistics efficiently.

Debugging should involve checking feature dimensions before and after selection, verifying that no target variable has accidentally been included as an input feature, and checking for duplicate or highly correlated features. The final feature matrix should also be checked for missing or infinite values.

15. You are required to integrate and analyze data from multiple IoT devices with varying sampling rates and missing timestamps. Constraints include: (i) aligning time-series data correctly, (ii) handling missing intervals without distortion, and (iii) maintaining computational efficiency. Explain how you will use **Pandas time-series functions**, reindexing, interpolation, and aggregation to enforce these constraints, and discuss debugging techniques for misaligned or inconsistent time-series data.

IoT devices may collect measurements at different sampling frequencies. Correct time alignment is therefore essential before combining their data.

Pandas provides powerful time-series functionality for this task. First, timestamps should be converted using `pd.to_datetime()` and set as the DataFrame index. Each device's records should then be sorted chronologically.

A common time frequency can be selected using `resample()`. For example, high-frequency measurements can be aggregated into one-minute averages, while lower-frequency data can be aligned to the same time interval.

Missing timestamps can be identified using `reindex()` with a complete expected time index. Missing observations should not automatically be replaced without considering the nature of the sensor. Interpolation may be appropriate for continuous measurements, while forward filling may be suitable for certain slowly changing states.

Aggregation can calculate mean, minimum, maximum, or sum values for each interval. The chosen method should match the physical meaning of the sensor measurement.

Debugging should verify that timestamps are correctly sorted, time zones are consistent, and expected intervals are present. Sample records should be manually checked before and after resampling. Sudden unexpected changes may indicate incorrect interpolation or aggregation. Performance can be improved by processing only required columns and using efficient time-series operations.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand